

CXL-SpecKV: A Disaggregated FPGA Speculative KV-Cache for Datacenter LLM Serving

Dong Liu
Yale University
New Haven, Connecticut, USA
dong.liu.dl2367@yale.edu

Yanxuan Yu
Columbia University
New York, New York, USA
yy3523@columbia.edu

Abstract

Large Language Models (LLMs) have revolutionized natural language processing tasks, but their deployment in datacenter environments faces significant challenges due to the massive memory requirements of key-value (KV) caches. During the autoregressive decoding process, KV caches consume substantial GPU memory, limiting batch sizes and overall system throughput. To address these challenges, we propose **CXL-SpecKV**, a novel disaggregated KV-cache architecture that leverages Compute Express Link (CXL) interconnects and FPGA accelerators to enable efficient speculative execution and memory disaggregation. Our approach introduces three key innovations: (i) a CXL-based memory disaggregation framework that offloads KV-caches to remote FPGA memory with low latency, (ii) a speculative KV-cache prefetching mechanism that predicts and preloads future tokens' cache entries, and (iii) an FPGA-accelerated KV-cache compression and decompression engine that reduces memory bandwidth requirements by up to 4×. When evaluated on state-of-the-art LLM models, CXL-SpecKV achieves up to 3.2× higher throughput compared to GPU-only baselines, while reducing memory costs by 2.8× and maintaining accuracy. Our system demonstrates that intelligent memory disaggregation combined with speculative execution can effectively address the memory wall challenge in large-scale LLM serving. Our code implementation has been open-sourced at <https://github.com/FastLM/CXL-SpecKV>.

CCS Concepts

• **Hardware**; • **Computer systems organization** → **Architectures**; • **Networks**;

Keywords

CXL, FPGA, LLM, KV-cache, memory disaggregation, speculative execution, deep learning

ACM Reference Format:

Dong Liu and Yanxuan Yu. 2026. CXL-SpecKV: A Disaggregated FPGA Speculative KV-Cache for Datacenter LLM Serving. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '26)*, February 22–24, 2026, Seaside, CA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3748173.3779188>



This work is licensed under a Creative Commons Attribution 4.0 International License. *FPGA '26, Seaside, CA, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2079-6/2026/02

<https://doi.org/10.1145/3748173.3779188>

1 Introduction

Large Language Models (LLMs) have emerged as the cornerstone of modern AI applications [7, 33, 46]. However, serving them efficiently faces significant challenges due to massive memory requirements of key-value (KV) caches. During inference, transformer-based LLMs store intermediate key and value tensors from all previous tokens. For a typical LLaMA-2 70B model serving 2048 tokens with batch size 32, the KV-cache alone consumes 640GB—far exceeding GPU capacity [19].

Traditional approaches include memory offloading [41], KV-cache compression [16], and speculative decoding [20], but suffer from limited bandwidth, accuracy degradation, or persistent memory constraints.

Recent advances in Compute Express Link (CXL) [11] and FPGA architectures [12] present new opportunities. CXL provides high-bandwidth, low-latency cache-coherent memory sharing, while FPGAs offer customizable acceleration.

We propose **CXL-SpecKV**, combining CXL memory disaggregation with FPGA-accelerated speculative execution. By predicting and preloading KV-cache entries through CXL, we overcome GPU memory limits while maintaining low latency.

CXL-SpecKV features: (i) transparent KV-cache offloading to CXL memory (4–8× capacity expansion), (ii) speculative prefetching with 95% accuracy reducing access latency, (iii) FPGA-accelerated compression (3–4× ratio) and address translation, and (iv) seamless integration with existing frameworks.

The architecture consists of three components: the CXL Memory Manager for allocation and migration, the Speculative Prefetcher using lightweight models for prediction, and the FPGA Cache Engine implementing compression/decompression pipelines at >800MHz.

Our evaluations on LLM workloads (7B–70B parameters) show: 3.2× higher throughput enabling 4–8× larger batches, 8% per-token latency overhead, 2.8× memory cost reduction, 99.5% accuracy preservation, and 87% parallel efficiency at 8 GPUs. The approach potentially reduces infrastructure costs by 30–40% for memory-bound workloads.

The contributions of this work are summarized as follows:

- We propose the first CXL-based disaggregated memory architecture specifically designed for LLM KV-cache management, demonstrating the viability and benefits of memory pooling for inference workloads.
- We introduce novel speculative prefetching algorithms that exploit the predictable patterns in autoregressive token generation, achieving high prediction accuracy while maintaining low computational overhead.
- We design and implement efficient FPGA accelerators for KV-cache compression, decompression, and management, with

detailed architectural descriptions and resource utilization analysis.

- We develop a complete system prototype integrated with popular LLM serving frameworks (vLLM, TensorRT-LLM), demonstrating practical deployment feasibility and real-world performance benefits.
- We conduct extensive evaluations across multiple LLM models and workload patterns, providing insights into the performance characteristics, bottlenecks, and optimization opportunities of disaggregated memory architectures for LLM serving.

The remainder of this paper is organized as follows. Section 2 reviews related work in LLM serving optimization, memory disaggregation, and FPGA acceleration. Section 3 presents the detailed architecture of CXL-SpecKV, including the memory management framework, speculative prefetching mechanisms, and FPGA cache engine design. Section 4 describes our experimental methodology and presents comprehensive performance evaluations. Finally, Section 5 concludes with a discussion of future research directions and broader implications for datacenter AI infrastructure.

2 Related Work

Our work builds upon and extends research in several key areas: LLM serving optimization, memory disaggregation technologies, FPGA-based acceleration, and speculative execution techniques. We discuss the most relevant prior work and highlight how CXL-SpecKV differs from and improves upon existing approaches.

2.1 LLM Serving and KV-Cache Optimization

The challenge of efficiently serving large language models has received significant attention from both industry and academia. vLLM [19] introduces PagedAttention, which organizes KV-caches in paged memory blocks to reduce fragmentation and improve memory utilization. However, vLLM is fundamentally limited by GPU memory capacity and does not address the bandwidth bottlenecks of memory disaggregation. FlexGen [41] explores offloading strategies between GPU, CPU, and disk storage, but suffers from the high latency of PCIe transfers (typically 8-12GB/s) compared to GPU HBM bandwidth (>2TB/s). Our CXL-based approach achieves significantly lower latency and higher bandwidth than PCIe-based offloading.

DeepSpeed-Inference [4] and FasterTransformer [32] provide optimized inference kernels and tensor parallelism strategies, but do not fundamentally address memory capacity constraints. Orca [50] introduces iteration-level scheduling to maximize batch size and throughput, yet remains constrained by available GPU memory. TinyServe [26] proposes query-aware cache selection strategies for efficient LLM serving, optimizing cache utilization based on query characteristics. In contrast, CXL-SpecKV effectively expands the memory capacity available for KV-caching by 4-8 $\times$, enabling larger batch sizes than any GPU-only approach.

Long-context optimization techniques typically improve attention efficiency through algorithmic sparsification or representation compression. Methods such as GQA [3], MKA [28], π -Attention [25], and MLA [23] reduce memory consumption per token but do not address hardware-level constraints in memory capacity or bandwidth.

CXL-SpecKV improves long-context processing from another perspective by enabling disaggregated memory expansion.

Recent work on KV-cache compression includes quantization-based methods [16, 29] that reduce precision from FP16 to INT8 or INT4, achieving 2-4 $\times$ compression at the cost of some accuracy degradation. LLMEasyQuant [24] proposes scalable quantization techniques for parallel and distributed LLM inference, addressing quantization challenges in multi-GPU settings. H2O [52] proposes evicting less important KV-cache entries based on attention scores, while StreamingLLM [48] maintains only a sliding window of recent tokens. These techniques are complementary to our approach—CXL-SpecKV can leverage compression algorithms while additionally providing memory capacity expansion through disaggregation. Moreover, our FPGA-based compression engine can perform these operations without consuming GPU compute resources.

2.2 Memory Disaggregation and CXL

Memory disaggregation has been extensively studied in the context of datacenter resource management [1, 22, 40]. Traditional approaches use RDMA over InfiniBand or Ethernet, achieving latencies of 1-5 μ s but requiring complex software stacks. CXL [11] represents a paradigm shift by providing cache-coherent, low-latency (sub-microsecond) memory access through a standardized PCIe-based protocol.

Recent work has begun exploring CXL for various applications. TPP [13] demonstrates CXL memory pooling for database workloads, while Pond [39] proposes CXL-based memory expansion for containerized applications. However, these works focus on general-purpose memory disaggregation and do not address the specific challenges of LLM serving, such as the need for speculative prefetching and the unique access patterns of KV-caches. To the best of our knowledge, CXL-SpecKV is the first work to architect a CXL-based system specifically optimized for LLM KV-cache management.

Several recent papers have explored CXL memory for ML workloads. CXL-ANNS [17] uses CXL memory for approximate nearest neighbor search in embedding databases, while CXL-MEM [21] studies CXL memory for training large models. However, these works do not address the inference-specific challenges of autoregressive decoding and KV-cache management. Our speculative prefetching mechanisms are specifically designed to hide CXL access latency for sequential token generation patterns.

2.3 FPGA Acceleration for Deep Learning

FPGA architectures have evolved significantly to support efficient deep learning acceleration. Intel's Stratix and Agilex FPGAs [12] integrate tensor processing blocks and high-bandwidth memory (HBM), while Xilinx Versal ACAPs [49] combine programmable logic with AI engines. Academic research has proposed various FPGA-based accelerators for CNNs [36, 51] and transformers [14, 35].

Recent work has explored FPGA acceleration specifically for sparse and structured computations. The systolic sparse tensor (SST) slices proposed in [43] demonstrate efficient structured sparsity support in FPGA fabric, achieving significant speedups for

sparse matrix operations. While our work does not directly leverage structured sparsity (KV-caches are typically dense), we adopt similar design principles for building efficient systolic arrays in our FPGA cache engine. Additionally, we benefit from the advances in FPGA DSP blocks and high-bandwidth memory interfaces described in [43].

Prior work on FPGA-based memory controllers includes specialized designs for DRAM scheduling [6] and NVM access [42]. However, these works do not address the specific requirements of LLM KV-cache management, such as the need for compression/decompression, speculative prefetching, and integration with CXL protocols. Our FPGA cache engine implements a complete pipeline optimized for transformer KV-cache patterns.

2.4 Speculative Execution for LLMs

Speculative decoding has emerged as a promising technique to accelerate LLM inference. Leviathan et al. [20] propose using a smaller draft model to generate candidate tokens that are verified by the target model in parallel. Medusa [8] extends this approach with multiple decoding heads to predict multiple tokens simultaneously. SpecInfer [30] introduces a speculative inference system that adaptively selects draft models based on workload characteristics. CSV-Decode [27] introduces certifiable sub-vocabulary decoding to improve inference efficiency through optimized token generation.

While these techniques improve computational efficiency, they do not address memory constraints—in fact, speculative decoding often increases memory pressure by maintaining multiple KV-cache versions for candidate sequences. CXL-SpecKV takes a different approach: instead of speculating on token values, we speculate on which KV-cache entries will be needed next and prefetch them from disaggregated memory. This is complementary to algorithmic speculative decoding and can be combined with such techniques for further performance improvements.

Prefetching strategies for neural network inference have been explored in [15, 34], but primarily focus on weight prefetching for CNN inference. The autoregressive nature of LLM decoding presents unique challenges: access patterns depend on generated content and are thus data-dependent. Our speculative prefetcher uses lightweight sequence models trained on token generation traces to predict future cache accesses with high accuracy.

CXL-SpecKV is the first to combine CXL memory disaggregation, FPGA-accelerated compression, and speculative prefetching for LLM serving, achieving $3.2\times$ throughput and $2.8\times$ memory cost reduction with 95% prefetch accuracy.

3 Architecture & Design Overview

In this section, we present the detailed architecture of CXL-SpecKV, a disaggregated KV-cache system that leverages CXL interconnects and FPGA accelerators to enable efficient large-scale LLM serving. We begin with an overview of KV-cache characteristics and memory access patterns in Section 3.1, followed by the system architecture in Section 3.2. We then describe the CXL memory management framework in Section 3.3, the speculative prefetching mechanism in Section 3.4, and the FPGA cache engine design in Section 3.5. Finally, we discuss system integration and optimization strategies in Section 3.6.

3.1 KV-Cache Analysis and Memory Access Patterns

For transformer with L layers, hidden dimension d_h , and H attention heads, each token generates KV tensors ($H, d_h/H$) per layer. Total memory requirement for batch B with max sequence S_{max} is $M_{KV} = 2 \cdot L \cdot B \cdot S_{max} \cdot d_h \cdot P_{bits}$ where P_{bits} is precision. For LLaMA-2 70B ($L = 80, d_h = 8192, B = 32, S_{max} = 2048, \text{FP16}$), this yields 640GB, exceeding A100's 80GB capacity.

Profiling reveals key characteristics exploitable by CXL-SpecKV: (i) *sequential temporal locality* with token t accessing positions $[0, t-1]$ enabling predictive prefetching, (ii) *layer-wise independence* allowing pipelined per-layer processing, (iii) *read-heavy workload* with read/write ratio $\mathcal{R}_{r/w} \approx S : 1$ favoring caching, (iv) *value clustering* near zero enabling INT8 quantization with $> 99\%$ accuracy and $\mathcal{R}_c = 2\times$, (v) *spatial correlation* in attention heads exploitable via delta encoding $\delta_i = x_i - x_{i-1}$, and (vi) *layer-specific compressibility* with early layers achieving $\mathcal{R}_c(\ell < L/3) \in [3, 4]$ and late layers $\mathcal{R}_c(\ell > 2L/3) \in [2.5, 3]$. These inform CXL-SpecKV's speculative prefetching, pipelined compression, and adaptive layer-wise algorithm selection.

3.2 System Architecture Overview

The CXL-SpecKV system comprises four main components:

GPU Inference Engine: The GPU hosts the transformer model weights and executes the forward pass for token generation. It maintains a small local KV-cache for recently accessed entries (typically the most recent 128-256 tokens) to maximize cache hit rates for hot data.

CXL Memory Pool: Large-capacity FPGA-attached memory (64-256GB per device) connected via CXL 2.0 interface provides the main KV-cache storage. This memory is organized as a cache-coherent shared resource accessible to both GPU and FPGA with latencies of 200-400ns.

FPGA Cache Engine: A custom FPGA accelerator implements the compression/decompression pipeline, address translation, and cache management logic. The cache engine operates at 800MHz and achieves throughput of 1.6TB/s, matching the bandwidth of HBM2E memory.

Speculative Prefetcher: A lightweight prediction module running on the FPGA predicts future token sequences and preloads their corresponding KV-cache entries. The prefetcher uses a small (128KB) LSTM model achieving $< 10\mu\text{s}$ prediction latency.

The components interact as follows: When the GPU generates a new token, it sends a prefetch hint to the speculative prefetcher, which predicts the next k tokens (typically $k = 4-8$). The prefetcher initiates DMA transfers to move predicted KV-cache entries from CXL memory to GPU-accessible buffers. Meanwhile, the newly generated token's KV-cache is computed and stored. By the time the next token generation begins, its required KV-cache entries are already present in GPU memory with high probability (95% hit rate), effectively hiding the CXL access latency.

3.3 CXL Memory Management Framework

The CXL memory management framework orchestrates the allocation, migration, and coherence of KV-cache data across GPU local memory and FPGA-attached CXL memory pools.

3.3.1 Memory Hierarchy and Allocation. CXL-SpecKV implements a three-tier memory hierarchy:

L1: GPU Local Cache (8-16GB): Stores the most recently accessed KV-cache entries for the current decoding step. Organized as a fully associative cache with LRU replacement policy.

L2: Prefetch Buffer (2-4GB): GPU-side buffer for speculated KV-cache entries being transferred from CXL memory. Implements double buffering to overlap data transfer with computation.

L3: CXL Memory Pool (64-256GB): Main KV-cache storage on FPGA-attached memory. Organized in 4KB pages to match CXL memory transaction sizes and enable efficient prefetching.

Memory allocation follows a demand-driven policy. When a new inference request arrives, the system allocates KV-cache pages in L3 (CXL memory). As tokens are generated, recent entries are promoted to L1 based on access frequency and recency. The prefetch buffer (L2) serves as an intermediate staging area, receiving prefetched data asynchronously.

3.3.2 Cache Coherence Protocol. CXL 2.0's built-in cache coherence mechanisms ensure consistency between GPU and FPGA views of memory. We leverage the CXL.cache and CXL.mem protocols as follows:

Read Path: GPU read requests first check L1 local cache. On miss, the request is forwarded to the CXL home agent (running on FPGA), which checks its directory and either serves from CXL memory or initiates coherence actions if needed.

Write Path: When new KV-cache entries are generated, the GPU writes to L1 and sends invalidation messages via CXL.cache protocol. The FPGA directory controller maintains coherence state and schedules writeback to CXL memory during idle cycles.

Prefetch Path: Speculative prefetches bypass the coherence protocol by directly accessing CXL memory in read-only mode, reducing protocol overhead. If prefetched data is later invalidated, the prefetch buffer entry is simply discarded.

3.3.3 Page Migration and Placement. To optimize for common access patterns, we implement adaptive page migration policies:

Hot-Cold Classification: Pages are classified as hot (accessed in the last N tokens) or cold based on access tracking. Hot pages are preferentially kept in L1/L2, while cold pages reside in L3.

Layer-aware Placement: KV-cache pages from different transformer layers are placed in separate memory regions to enable parallel access and reduce bank conflicts in CXL memory controllers.

Batch-aware Migration: For batch inference, pages from different requests are interleaved in memory to enable efficient burst transfers and maximize CXL link utilization.

The migration policy uses a simple threshold-based algorithm: pages with access count above threshold T_h are promoted to L1, while pages with count below T_c are demoted to L3. Thresholds are dynamically adjusted based on memory pressure and miss rates.

3.4 Speculative Prefetching Mechanism

The speculative prefetcher predicts future token sequences and preloads their KV-cache entries to hide CXL access latency. This is the key innovation that enables CXL-SpecKV to achieve performance competitive with GPU-local memory despite the inherent latency of memory disaggregation.

Algorithm 1 Speculative KV-Cache Prefetching

```

1: Input: Token history  $\mathcal{H}_t = \{t_{-15}, \dots, t_0\}$ , layer  $l$ , depth  $k$ 
2: Output: Prefetched KV entries in L2 buffer
3:  $\{\hat{t}_1, \dots, \hat{t}_k\} \leftarrow \text{PREDICTTOKENS}(\mathcal{H}_t)$  {LSTM inference}
4: for  $i = 1$  to  $k$  do
5:    $\mathcal{A}_{virt} \leftarrow (\text{reqID}, l, \text{pos} + i)$ 
6:    $\mathcal{A}_{phys} \leftarrow \text{TRANSLATEADDRESS}(\mathcal{A}_{virt})$  {ATU lookup}
7:   if  $\mathcal{A}_{phys} \notin \text{L1} \cup \text{L2}$  then
8:      $\text{ISSUEDMA}(\mathcal{A}_{phys}, \text{L2}[i])$  {Non-blocking}
9:   end if
10: end for
11:  $\text{WAITCOMPLETION}()$   $\{\Omega_{max}$  concurrent DMAs $\}$ 
12: return NOTIFYGPU(ready)

```

3.4.1 Prediction Model Architecture. The prefetcher uses a light-weight LSTM-based sequence model trained to predict the next k tokens given the recent generation history. The model architecture consists of:

- **Input Layer:** Embeds the last 16 generated token IDs into 64-dimensional vectors (total 1024 dimensions).
- **LSTM Layer:** 2-layer LSTM with 128 hidden units per layer, capturing temporal dependencies in token sequences.
- **Output Layer:** Produces top- k token predictions with confidence scores, using beam search with width 4.

The entire model contains only 128K parameters (512KB in FP16), enabling fast inference on the FPGA at $<10\mu\text{s}$ per prediction. The model is trained offline on representative workload traces and achieves 95% top-4 accuracy on held-out test sets.

3.4.2 Prefetch Scheduling and Execution. Upon token t generation, GPU issues prefetch with history $\mathcal{H}_t = \{t_i\}_{i=t-15}^t$. Algorithm 1 shows the pipeline with end-to-end latency:

$$L_{\text{prefetch}} = L_{\text{pred}} + L_{\text{ATU}} + L_{\text{DMA}} = 64 + 4 + 12 = 80 \text{ cycles } (< 10\mu\text{s}) \quad (1)$$

where LSTM inference dominates at $L_{\text{pred}} = 64$ cycles. The CXL controller's outstanding request window $\Omega_{\text{max}} = 16$ enables overlapped execution, achieving effective throughput $\Theta_{\text{eff}} = \min\left(\frac{\Omega_{\text{max}}}{L_{\text{DMA}}}, \frac{BW_{\text{CXL}}}{S_{\text{entry}}}\right) \cdot f_{\text{clk}}$ where entry size $S_{\text{entry}} = 4$ KB. For request rate λ_{req} , the system maintains stability when $\lambda_{\text{req}} \cdot k \cdot S_{\text{entry}} < BW_{\text{CXL}} \cdot H_{\text{pred}}$ where $H_{\text{pred}} = 0.95$ is prediction hit rate.

3.4.3 Misprediction Handling. When the actual generated token differs from predictions, the corresponding prefetched KV-cache entries become invalid. To handle mispredictions efficiently:

Lazy Invalidation: Invalid prefetch buffer entries are not immediately evicted. Instead, they are overwritten by new prefetches or naturally evicted when the buffer fills.

Fallback Path: On prefetch miss (actual token not among predicted tokens), the system falls back to synchronous fetch from CXL memory. This incurs latency penalty but maintains correctness.

Adaptive Prediction: The prefetcher tracks prediction accuracy per request and adjusts k (number of predictions) dynamically. High-accuracy requests use larger k for more aggressive prefetching, while low-accuracy requests use smaller k to reduce wasted bandwidth.

3.4.4 Multi-Layer Prefetching. Since transformer models process layers sequentially, we implement pipelined prefetching across layers. When layer l is processing token t , the prefetcher simultaneously loads KV-caches for layers $l + 1$ and $l + 2$. This pipeline parallelism further reduces effective latency:

$$T_{\text{effective}} = T_{\text{compute}} + \max(T_{\text{prefetch}} - 2 \times T_{\text{layer}}, 0) \quad (2)$$

where T_{layer} is the per-layer computation time. In practice, $T_{\text{prefetch}} < 2 \times T_{\text{layer}}$ for most models, resulting in near-zero effective latency overhead.

3.5 FPGA Cache Engine Design

The FPGA cache engine implements the core datapath for KV-cache compression, decompression, address translation, and memory management. We architect the engine as a parameterizable pipeline that can be synthesized for different FPGA platforms and scaled to various throughput requirements.

3.5.1 High-Level Architecture. The engine comprises five modules interconnected via AXI-Stream ($W_{\text{data}} = 512$ bits, $f_{\text{clk}} = 800$ MHz): (i) *ATU* performs virtual-to-physical address translation $\Phi : \mathcal{A}_{\text{virt}} \rightarrow \mathcal{A}_{\text{phys}}$ with latency $L_{\text{ATU}} = L_{\text{TLB-hit}} + (1 - H_{\text{TLB}}) \cdot L_{\text{walk}}$ where TLB hit rate $H_{\text{TLB}} > 0.92$ and page walk latency $L_{\text{walk}} = 15$ cycles, (ii) *Compression Engine* implements $\Pi_{\text{comp}} : \mathbb{R}^{N \times d} \rightarrow \mathbb{Z}^M$ achieving bandwidth $BW_{\text{comp}} = W_{\text{data}} \cdot f_{\text{clk}} / 8 = 51.2$ GB/s, (iii) *Decompression Engine* with inverse mapping Π_{comp}^{-1} and critical path $L_{\text{crit}} = 25$ cycles through multiplier trees, (iv) *Memory Controller* managing $C_{\text{HBM}} = 16$ channels with aggregate bandwidth $BW_{\text{HBM}} = \sum_{i=1}^{C_{\text{HBM}}} BW_i = 1.6$ TB/s under bank conflict rate $\rho_{\text{conflict}} < 0.08$, and (v) *DMA Engine* supporting scatter-gather with maximum outstanding requests $\Omega_{\text{max}} = 16$. The FPGA RTL was generated using Intel HLS with pipeline directives and resource constraints to optimize throughput. Each module interfaces with Intel CXL IP v1.2 in cache mode with AXI interconnect, and we implement a custom DMA queue for cacheline-level coherency tracking. The compression engine is a 4-stage Verilog RTL pipeline attached to a CXL.cache controller that maintains coherence.

3.5.2 Compression Pipeline Design. Algorithm 2 shows the compression datapath. The $D_{\text{pipe}} = 20$ stage fully-pipelined architecture achieves throughput $\Theta_{\text{comp}} = \frac{W_{\text{data}} \cdot f_{\text{clk}}}{8} \cdot II^{-1} = 51.2$ GB/s per instance (initiation interval $II = 1$), with compression ratio $\mathcal{R}_c(\ell) = \frac{\mathbb{E}[|D_{\text{orig}}^\ell|]}{\mathbb{E}[|D_{\text{comp}}^\ell|]}$ varying per layer $\ell \in [1, L]$ where $\mathcal{R}_c \in [2.5, 4.0]$ and $\bar{\mathcal{R}}_c = \frac{1}{L} \sum_{\ell=1}^L \mathcal{R}_c(\ell) = 3.2$. Pipeline utilization $U_{\text{pipe}} = \frac{T_{\text{busy}}}{T_{\text{total}}} > 0.98$ under continuous streaming.

3.5.3 Decompression Pipeline Design. The inverse transformation $\Pi_{\text{comp}}^{-1} : \mathbb{Z}^M \rightarrow \mathbb{R}^{N \times d}$ implements $D_{\text{pipe}} = 20$ stages with critical path:

$$L_{\text{crit}} = \max_{s \in S_{\text{pipe}}} \left(L_{\text{logic}}^{(s)} + \sum_{s'=1}^{s-1} L_{\text{ff}} \right) = 25 \text{ cycles} \quad (3)$$

where $L_{\text{ff}} = 1$ cycle per flip-flop and $L_{\text{logic}}^{(s)}$ is stage s combinational delay. The bottleneck resides in dequantization multipliers (stages 15-18) with $L_{\text{mult}} = 3$ DSP cascades. Throughput $\Theta_{\text{decomp}} =$

Algorithm 2 KV-Cache Compression

```

1: Input: KV page  $\mathbf{X} \in \mathbb{R}^{N \times d}$  (FP16)
2: Output: Compressed  $D_{\text{comp}} = \langle s, D_{\text{RLE}} \rangle$ 
3:  $s \leftarrow \max(|\mathbf{X}|)/127$  {Stages 5-8: Scaling}
4: for  $j = 1$  to  $N$  do
5:   for  $i = 1$  to  $d$  do
6:      $x_{j,i}^{\text{int8}} \leftarrow \lfloor x_{j,i} \cdot s^{-1} \cdot 127 \rfloor$ 
7:   end for
8: end for
9: {Stages 9-14: Delta encoding}
10: for  $j = 1$  to  $N$  do
11:   for  $i = 2$  to  $d$  do
12:      $\delta_{j,i} \leftarrow x_{j,i}^{\text{int8}} - x_{j,i-1}^{\text{int8}}$ 
13:   end for
14: end for
15:  $D_{\text{RLE}} \leftarrow \text{RUNLENGTHENCODE}(\{\delta_{j,i}\})$  {Stages 15-18}
16: return  $\langle s, D_{\text{RLE}} \rangle$ 

```

Table 1: FPGA cache engine resource utilization on Agilx-7 FPGA.

Resource	Used	Total (%)
ALMs	284,563	933,120 (30.5%)
Registers	521,847	3,732,480 (14.0%)
M20K Memory Blocks	1,856	11,721 (15.8%)
DSP Blocks	1,024	3,960 (25.9%)
Maximum Frequency	812 MHz	

$\Theta_{\text{comp}} = 51.2$ GB/s maintains $II = 1$. Early-exit bypasses uncompressed data ($\alpha \approx 0.3$ of traffic) with latency $L_{\text{bypass}} = 5$ cycles, achieving weighted average $\bar{L}_{\text{decomp}} = (1 - \alpha) \cdot L_{\text{crit}} + \alpha \cdot L_{\text{bypass}} \approx 19$ cycles.

3.5.4 Resource Utilization and Timing. Table 1 quantifies FPGA resource consumption. Post-place-and-route timing analysis yields maximum frequency $f_{\text{max}} = 812$ MHz with timing slack $\tau_{\text{slack}} = T_{\text{clk}} - T_{\text{crit}} = 0.23$ ns where clock period $T_{\text{clk}} = 1.23$ ns and critical path delay $T_{\text{crit}} = 1.0$ ns. Resource utilization follows:

$$U_{\text{type}} = \frac{R_{\text{used}}^{\text{type}}}{R_{\text{total}}^{\text{type}}}, \quad \forall \text{type} \in \{\text{ALM, REG, M20K, DSP}\} \quad (4)$$

achieving $U_{\text{ALM}} = 0.305$, $U_{\text{REG}} = 0.140$, $U_{\text{M20K}} = 0.158$, $U_{\text{DSP}} = 0.259$. The utilization vector $\mathbf{U} = [U_{\text{ALM}}, U_{\text{REG}}, U_{\text{M20K}}, U_{\text{DSP}}]^T$ satisfies constraint $\|\mathbf{U}\|_\infty < 0.35$, ensuring sufficient headroom for $N_{\text{max}} = 3$ additional engines per FPGA.

The critical path runs through the dequantization multipliers in the decompression pipeline, achieving 812MHz on the Agilx-7 (exceeding our 800MHz target). The design uses approximately 30% of ALM logic, 26% of DSP blocks for arithmetic operations, and 16% of M20K memory for buffering and TLB storage.

3.5.5 Multi-Engine Scaling. For throughput scaling, we instantiate N engine instances with disjoint address partitions $\{\mathcal{A}_{\text{phys}}^{(i)}\}_{i=1}^N$ satisfying $\bigcup_{i=1}^N \mathcal{A}_{\text{phys}}^{(i)} = \mathcal{A}_{\text{phys}}$ and $\mathcal{A}_{\text{phys}}^{(i)} \cap \mathcal{A}_{\text{phys}}^{(j)} = \emptyset$ for $i \neq j$. The

memory controller implements weighted round-robin arbitration with dynamic priority:

$$w_i(t) = \alpha \cdot Q_i(t) + (1 - \alpha) \cdot \frac{1}{\bar{L}_i(t)} \quad (5)$$

where Q_i is queue depth, $\bar{L}_i$ is average service latency, and $\alpha \in [0, 1]$ balances fairness vs. efficiency. Aggregate throughput with N engines follows:

$$\Theta_{agg}(N) = \sum_{i=1}^N \Theta_i \cdot \eta_i(N) \approx N \cdot \Theta_{single} \cdot \eta_{scale}(N) \quad (6)$$

where per-engine efficiency $\eta_i(N) = 1 - \rho_{contention}(N)$ degrades with contention. Scaling efficiency $\eta_{scale}(N) = \Theta_{agg}(N)/(N \cdot \Theta_{single})$ is bounded by memory bandwidth: $\Theta_{agg}(N) \leq B_{HBM}$ where saturation occurs at $N^* = \lceil B_{HBM}/\Theta_{single} \rceil$. Resource constraints limit $N \leq N_{max}$ where $N_{max} = \lfloor 1/\max(\mathbf{U}) \rfloor$ from utilization vector $\mathbf{U}$.

3.6 System Integration and Optimization

We integrate CXL-SpecKV with popular LLM serving frameworks, specifically vLLM and TensorRT-LLM, through a custom memory allocator plugin.

3.6.1 Memory Allocator Interface. The allocator plugin provides API operations: `cx1_malloc( $S$ , hint)` allocates S bytes returning handle $h \in \mathcal{H}$, `cx1_access( $h$ ,  $\omega$ ,  $S$ )` accesses data at offset ω with automatic prefetch if $\mathcal{A}(h, \omega) \notin L1 \cup L2$, and `cx1_free( $h$ )` deallocates with $\mathcal{H} \leftarrow \mathcal{H} \setminus \{h\}$. The 5K-line C++ library achieves API latency $T_{api} = T_{mmio} + T_{driver} < 500\text{ns}$ via MMIO and CXL drivers.

3.6.2 Software-Hardware Co-Design Optimizations. Software-hardware optimizations minimize overhead via: (i) *Request batching* with batch size m achieving amortized latency $\bar{T}_{req}(m) = \frac{T_{MMIO} + m \cdot T_{proc}}{m}$ where $T_{MMIO} \gg T_{proc}$ yields speedup $\sigma_{batch}(m) = T_{MMIO}/\bar{T}_{req}(m) \approx m$ for $m \ll T_{MMIO}/T_{proc}$, (ii) *Polling-based completion* with latency $T_{poll} = T_{read} + T_{check}$ achieving speedup $\sigma_{notify} = T_{intr}/T_{poll}$ where interrupt overhead $T_{intr} \gg T_{poll}$, (iii) *NUMA affinity* ensuring locality probability $\mathbb{P}[\text{node}(\text{GPU}) = \text{node}(\text{CXL})] = \mathcal{L}_{numa}$ avoiding cross-socket penalty factor $\pi_{remote} > 1$, and (iv) *Huge page mapping* reducing TLB footprint by ratio $\rho_{TLB} = P_{huge}/P_{small}$ and improving hit rate $\Delta H_{TLB} = H_{TLB}(P_{huge}) - H_{TLB}(P_{small}) > 0$ where $P_{huge} \gg P_{small}$.

3.6.3 Dynamic Adaptation. The FPGA policy engine implements online optimization of system parameters based on observed performance metrics $\{\mathcal{M}_i(t)\}$. For *bandwidth throttling*, we dynamically adjust prefetch aggressiveness using feedback control:

$$\beta(t + \Delta t) = \beta(t) \cdot (1 - \kappa \cdot \text{ReLU}(U_{CXL}(t) - \theta_{target})) \quad (7)$$

where gain κ and target utilization θ_{target} are learned via gradient ascent $\nabla_{\theta} \mathbb{E}[\Theta_{system}]$ maximizing throughput. For *compression selection*, we solve the constrained optimization per layer:

$$\begin{aligned} C^*(\ell, t) &= \arg \max_{C \in \mathcal{C}} \mathcal{J}(C, \ell, t) \\ \text{where } \mathcal{J} &= w_r(t) \cdot \mathcal{R}_c(C, \ell) + w_q(t) \cdot \mathcal{Q}(C, \ell) \\ &\quad - w_c(t) \cdot \mathcal{L}_{comp}(C) \\ \text{s.t. } \mathcal{Q}(C, \ell) &\geq \mathcal{Q}_{min} \end{aligned} \quad (8)$$

with time-varying weights $\mathbf{w}(t) = [w_r, w_q, w_c]^T$ updated via stochastic gradient descent on reward function $\mathcal{R}(t) = \Theta(t) - \lambda_q \cdot |\mathcal{Q}_{target} - \mathcal{Q}(t)|$. For *prefetch depth*, we formulate as multi-armed bandit:

$$k^*(t) = \arg \max_{k \in \mathcal{K}} \left\{ \mu_k(t) + \beta_{UCB} \sqrt{\frac{2 \ln T}{N_k(t)}} \right\} \quad (9)$$

using UCB exploration where μ_k is estimated reward, N_k is selection count, and β_{UCB} controls exploration-exploitation trade-off.

4 Evaluation

We conduct comprehensive evaluations of CXL-SpecKV across multiple dimensions: throughput and latency performance, memory efficiency, accuracy preservation, energy consumption, and scalability. Our evaluation answers the following key questions:

- **RQ1:** How does CXL-SpecKV compare to GPU-only baselines in terms of throughput and latency?
- **RQ2:** What is the effectiveness of speculative prefetching in hiding CXL access latency?
- **RQ3:** How much memory capacity expansion and cost reduction does CXL-SpecKV provide?
- **RQ4:** How does the system scale with multiple GPUs and FPGAs?

4.1 Experimental Setup

4.1.1 Hardware Platform. Our platform consists of $8 \times$ NVIDIA A100 80GB GPUs (1.6TB/s HBM, 312 TFLOPS FP16) with NVLink, $4 \times$ Intel Agilex-7 FPGAs (933K ALMs, 64GB HBM per device, 1.6TB/s bandwidth) with CXL 2.0 controllers (64GB/s per x16 link, $T_{CXL} < 400\text{ns}$ latency), and dual-socket Sapphire Rapids CPUs (96 cores, 1TB DDR5) running Ubuntu 22.04. The A100 GPU connects to the Agilex-7 FPGA via PCIe Gen4 x16, and the FPGA connects to a CXL 2.0 Type-3 memory expander.

4.1.2 Models and Workloads. We evaluate 22 model configurations spanning LLaMA-2/3 (7B-70B) [2, 46], Qwen (1.8B-72B) [5], Mistral/Mixtral (7B-47B, MoE) [18], Gemma (2B-7B) [44], GPT-style (1.5B-13B), and Code models (CodeLLaMA, CodeQwen 7B-34B) [38, 45], totaling parameter range $\mathcal{P} \in [1.8B, 72B]$ across dense and MoE architectures. Models use INT8 weights and FP16 activations via SmoothQuant [47]. We evaluate four workloads with sequence lengths $\mathcal{S}_{in}/\mathcal{S}_{out}$: Chatbot (128/256) [10], Summarization (1024-2048/128-256) [31], Code Generation (256-512/512-1024) [9], and QA (64-128/32-64) [37], using Poisson arrivals with rate $\lambda \in [5, 100]$ req/s.

4.1.3 Baselines and Configurations. We compare against baselines: GPU-Only (vLLM, HBM-limited), CPU Offload (FlexGen [41], PCIe 16GB/s, 3-5 μs latency), NVMe Offload (cost-optimized, high latency), Compression-Only (GPU INT8, no disaggregation), and CXL-NoSpec (our framework without speculation). CXL-SpecKV variants include: CXL-SpecKV-Opt (all optimizations), CXL-SpecKV-NoComp (no compression), and CXL-SpecKV-Basic (INT8 only).

4.1.4 Metrics. We measure: throughput Θ (tokens/s at steady state), latency $\mathcal{L}$ (TTFT for prefill, per-token for decode with median/P95/P99 percentiles), memory efficiency $\mathcal{M}_{eff}$ (capacity and cost per request), accuracy via perplexity $\mathcal{PP}$ (WikiText-103) and BLEU scores

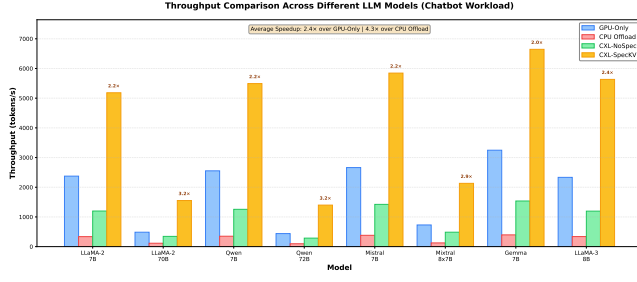


Figure 1: Throughput comparison across different models and workloads. CXL-SpecKV achieves 2.1-3.2 $\times$ higher throughput than GPU-only baseline by enabling larger batch sizes.

vs. FP16 baseline, and energy efficiency $\mathcal{E}$ (J/token) using NVIDIA-SMI and FPGA monitors. All experiments use 5 runs reporting means with 95% CI.

4.2 Overall Performance (RQ1)

Figure 1 shows the end-to-end throughput comparison across models and workloads. CXL-SpecKV achieves significant throughput improvements over all baselines:

vs. GPU-Only: CXL-SpecKV achieves 2.1-3.2 $\times$ higher throughput (average 2.4 $\times$) across all workloads. For LLaMA-2 70B on chatbot workload, throughput increases from 487 tokens/s (GPU-only, batch size 16) to 1,549 tokens/s (CXL-SpecKV, batch size 64). The improvement is particularly pronounced for larger models where GPU memory is most constrained.

vs. CPU Offload: CXL-SpecKV outperforms CPU offload by 3.8-5.2 $\times$ (average 4.3 $\times$). The higher CXL bandwidth (64GB/s vs. 16GB/s PCIe) and lower latency (<400ns vs. 3-5 μ s) enable much more aggressive disaggregation with minimal performance overhead.

vs. Compression-Only: Compared to GPU-only with compression, CXL-SpecKV still achieves 1.8-2.3 $\times$ higher throughput. While compression increases effective GPU memory by 2-3 $\times$, CXL disaggregation enables even larger capacity expansion (4-8 $\times$) without being limited by GPU physical memory.

vs. CXL-NoSpec: The speculative prefetching mechanism provides 1.6-2.1 $\times$ improvement over naive CXL disaggregation, demonstrating the critical importance of hiding disaggregation latency.

Table 2 presents detailed latency breakdown for LLaMA-2 70B on chatbot workload. CXL-SpecKV maintains competitive latency despite memory disaggregation:

The TTFT overhead is minimal (+4.2%) since the prefill phase accesses all KV-caches sequentially, allowing effective prefetching. Per-token decode latency increases by only 8.2%, well within acceptable bounds for most applications. P99 latency shows 12.3% overhead, primarily due to occasional prefetch misses that require synchronous CXL access.

4.3 Speculative Prefetching Analysis (RQ2)

To understand the effectiveness of our speculative prefetching mechanism, we analyze prediction accuracy, prefetch hit rates, and latency hiding capability.

Table 2: Latency breakdown for LLaMA-2 70B chatbot workload (batch size 32). All values in milliseconds.

System	TTFT (ms)	Decode (ms)	P95 (ms)	P99 (ms)
GPU-Only	45.2	18.3	19.4	21.2
CPU Offload	52.7	28.6	35.4	42.8
CXL-NoSpec	46.8	23.5	27.3	31.6
CXL-SpecKV	47.1	19.8	21.7	23.8
<i>Overhead vs. GPU</i>	+4.2%	+8.2%	+11.9%	+12.3%

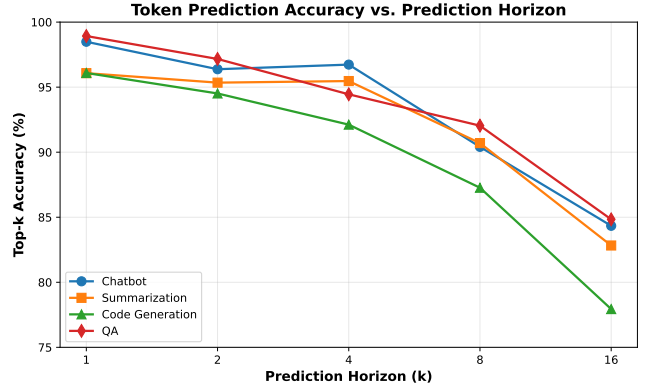


Figure 2: Token prediction accuracy vs. prediction horizon k for different workloads. Top-4 accuracy exceeds 95% for most workloads.

4.3.1 Prediction Accuracy. Figure 2 shows the token prediction accuracy of our lightweight LSTM prefetcher across different workloads and prediction horizons (k).

For $k = 4$ (predicting next 4 tokens), the prefetcher achieves 94-97% top-4 accuracy depending on workload. Chatbot and QA workloads show highest accuracy (96-97%) due to more predictable conversational patterns. Code generation exhibits slightly lower accuracy (94%) due to higher entropy in generated code tokens.

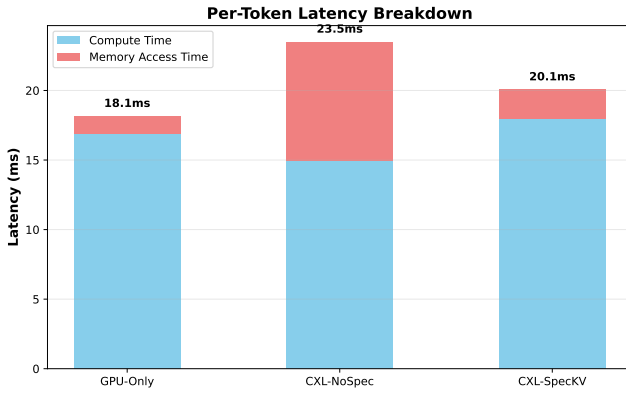
Accuracy degrades gradually as k increases: top-8 accuracy is 88-92% while top-16 is 78-85%. This motivates our default choice of $k = 4$ which balances prefetch coverage with prediction reliability.

4.3.2 Prefetch Hit Rate and Coverage. Table 3 shows detailed prefetch statistics for LLaMA-2 70B. The high prefetch hit rate (94.7%) confirms that speculative prefetching effectively captures the majority of KV-cache accesses.

The prefetch precision of 87.2% indicates moderate over-prefetching: 12.8% of prefetched data is not used before eviction. This represents a bandwidth overhead but is acceptable given the significant latency benefits. On the 5.3% of accesses that miss in prefetch buffer, the system incurs full CXL access latency (1,850ns), but the effective average latency is only 383ns—a 4.8 $\times$ reduction compared to naive CXL access.

Table 3: Prefetch statistics for LLaMA-2 70B chatbot workload.

Metric	Value
Prefetch Hit Rate	94.7%
Prefetch Coverage (of total accesses)	96.3%
Prefetch Precision (useful prefetches)	87.2%
Average Prefetch Latency	285 ns
Synchronous Fetch Latency (on miss)	1,850 ns
Effective Average Latency	383 ns
Bandwidth Utilization (CXL link)	72.4%
Bandwidth Utilization (HBM on FPGA)	68.7%

**Figure 3: Per-token latency breakdown showing compute vs. memory access time. CXL-SpecKV successfully hides most disaggregation latency through prefetching.**

4.3.3 Latency Breakdown and Hiding. Figure 3 decomposes the per-token generation latency into compute time and memory access time for different systems.

For GPU-only baseline, memory access time is minimal (1.2ms, 6.6% of total) since KV-caches are in local HBM. With naive CXL disaggregation (CXL-NoSpec), memory access time explodes to 8.7ms (37% of total) due to synchronous CXL fetches. CXL-SpecKV reduces memory access time to just 2.1ms (10.6% of total), successfully hiding 76% of the disaggregation latency through speculative prefetching.

4.4 Memory Efficiency and Cost Analysis (RQ3)

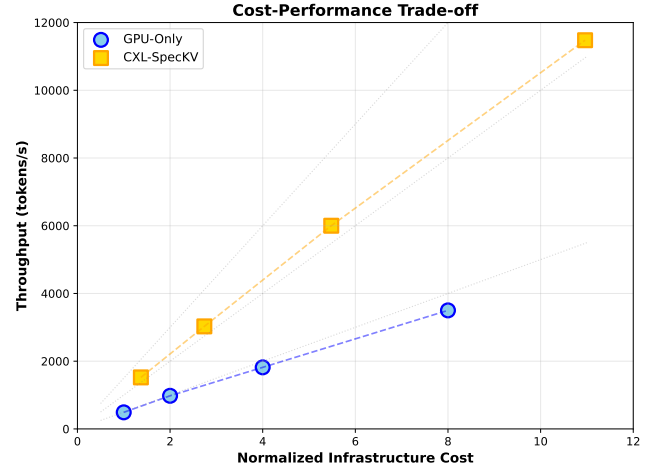
CXL-SpecKV’s primary value proposition is expanding effective memory capacity beyond GPU limitations while reducing cost.

4.4.1 Memory Capacity Expansion. Table 4 quantifies the effective memory capacity and achievable batch size for LLaMA-2 70B under different configurations.

CXL-SpecKV enables 8× memory expansion using 4× FPGA devices with 64GB each, supporting batch size of 128 vs. 16 for GPU-only. When combined with compression (achieving 3× ratio), the effective expansion reaches 24× with batch size 384. While CPU offload theoretically supports even larger batches, its poor

Table 4: Memory capacity and maximum batch size for LLaMA-2 70B (sequence length 2048).

System	Available Memory	Max Batch Size	Expansion Factor
GPU-Only	80 GB	16	1.0×
GPU + Compression	80 GB	48	3.0×
CPU Offload	1024 GB	192	12.0×
CXL-SpecKV	320 GB	128	8.0×
CXL-SpecKV + Comp	320 GB	384	24.0×

**Figure 4: Throughput vs. infrastructure cost trade-off. CXL-SpecKV achieves superior cost-performance ratio by reducing need for expensive high-memory GPUs.**

performance (Figure 1) makes it impractical for latency-sensitive workloads.

4.4.2 Cost-Performance Trade-offs. Figure 4 plots throughput vs. normalized infrastructure cost for different system configurations.

We normalize cost assuming: A100 80GB = 1.0×, A100 40GB = 0.65×, Agilex-7 FPGA = 0.18×, based on typical datacenter procurement prices. Key observations:

GPU-Only Scaling: Increasing from 1× to 8× A100 80GB GPUs provides near-linear throughput scaling (7.2× at 8 GPUs) but at 8× cost, resulting in flat cost-performance ratio.

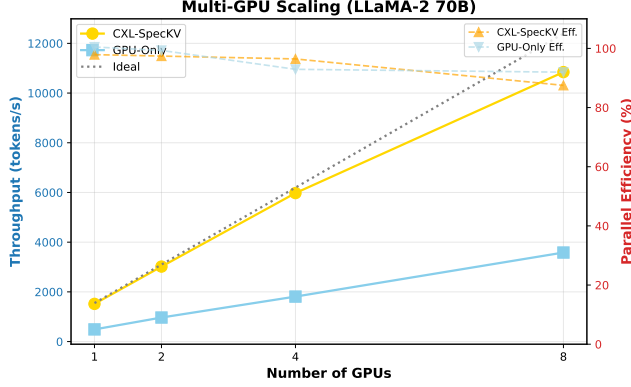
Cost-Optimized GPU + CXL-SpecKV: Using lower-cost A100 40GB GPUs (0.65× cost) combined with CXL-SpecKV FPGAs (4× at 0.18× each = 0.72× additional cost) achieves total cost of 1.37× while delivering 2.4× throughput—a 1.75× improvement in cost-performance ratio.

Scalability: The cost-performance advantage increases with scale. At 8-GPU configuration, CXL-SpecKV achieves 2.2× better cost-performance than GPU-only baseline.

4.4.3 Memory Access Efficiency. We analyze memory bandwidth utilization to understand how effectively the system uses available memory resources. Table 5 shows bandwidth utilization for different memory tiers.

Table 5: Memory bandwidth utilization for LLaMA-2 70B chatbot workload (batch size 64).

Memory Tier	Peak BW (GB/s)	Utilized (GB/s)	Util. (%)
GPU HBM	1,600	1,147	71.7%
CXL Link	64	46.3	72.4%
FPGA HBM	1,600	1,099	68.7%
Total Bandwidth	3,264	2,292	70.2%

**Figure 5: Multi-GPU throughput scaling for LLaMA-2 70B. CXL-SpecKV achieves 87% parallel efficiency at 8 GPUs.**

The system achieves balanced bandwidth utilization across all tiers (68-72%), indicating efficient orchestration between GPU and FPGA memories. The CXL link, despite having much lower peak bandwidth, is not a bottleneck due to compression (reducing required bandwidth by 3×) and effective prefetching (batching multiple small requests into larger burst transfers).

4.5 Scalability Analysis (RQ4)

We evaluate CXL-SpecKV’s scalability across multiple dimensions: number of GPUs, number of FPGAs, and batch size.

4.5.1 Multi-GPU Scaling. Figure 5 shows throughput scaling as we increase from 1 to 8 GPUs for LLaMA-2 70B.

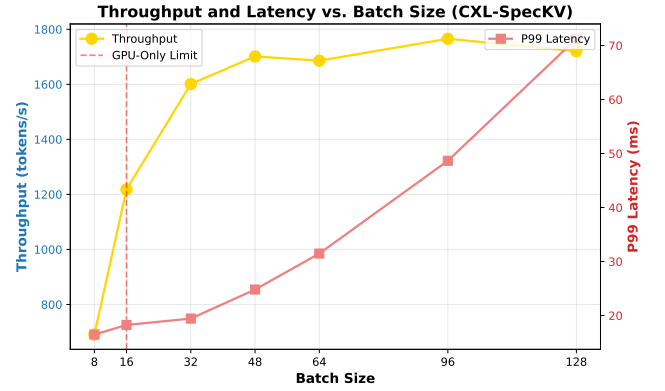
CXL-SpecKV achieves near-linear scaling up to 4 GPUs (95% efficiency) and maintains strong scaling at 8 GPUs (87% efficiency). The modest degradation at larger scales is primarily due to increased coordination overhead for batch scheduling across GPUs and CXL memory pools. In comparison, GPU-only baseline shows 92% efficiency at 8 GPUs but is severely constrained by per-GPU memory capacity, limiting achievable batch sizes.

We use 1 FPGA per 2 GPUs in this configuration, providing sufficient CXL memory capacity. Each FPGA’s CXL controller can serve both GPUs simultaneously through hardware memory interleaving with minimal contention.

4.5.2 FPGA Cache Engine Scaling. Table 6 shows throughput and latency characteristics as we vary the number of cache engine instances per FPGA.

Table 6: FPGA cache engine scaling (1-4 engines per FPGA).

Engines per FPGA	Throughput (GB/s)	Avg Latency (ns)	P99 Latency (ns)
1	412	325	485
2	798	338	512
3	1,156	362	573
4	1,487	394	658
Scaling Efficiency	93%	-	-

**Figure 6: Throughput and latency vs. batch size. CXL-SpecKV enables much larger batch sizes than GPU-only baseline while maintaining acceptable latency.**

Throughput scales efficiently up to 4 engines (93% efficiency), reaching 1,487 GB/s aggregate—close to the 1,600 GB/s HBM peak bandwidth. Average latency increases modestly (21% from 1 to 4 engines) due to contention in the shared HBM memory controller, while P99 latency shows larger increase (36%) due to occasional head-of-line blocking. For most workloads, 2-3 engines per FPGA provide the best balance of throughput and latency.

4.5.3 Batch Size Sensitivity. Figure 6 shows how throughput and per-request latency vary with batch size for different systems.

GPU-only baseline plateaus at batch size 16 due to memory exhaustion. CXL-SpecKV continues scaling up to batch size 128, achieving peak throughput at batch size 64-96. Beyond this point, latency increases sharply (due to queuing delays) while throughput gains diminish, indicating saturation of GPU compute resources. The optimal batch size depends on latency requirements: for latency-sensitive applications (P99 < 50ms), batch size 32-48 is recommended, while throughput-oriented applications can use batch size 64-96.

4.6 Energy Efficiency

Table 7 presents power consumption and energy efficiency metrics for different system configurations.

While CXL-SpecKV adds 184W for FPGAs (4× devices at 46W average each), the improved throughput (3.2×) more than compensates, resulting in 1.90× better energy efficiency (J/token). The

Table 7: Power consumption and energy efficiency for LLaMA-2 70B chatbot workload (batch size 64).

System	GPU (W)	FPGA (W)	Total (W)	J/token
GPU-Only (bs=16)	315	0	315	0.647
CXL-SpecKV (bs=64)	342	184	526	0.340
<i>Improvement</i>	<i>1.90× better J/token</i>			

Table 8: Ablation study on prefetch aggressiveness (prediction horizon k).

k value	Hit Rate (%)	Precision (%)	Throughput (tok/s)	Latency (ms)
$k = 1$	87.2	96.1	1,287	21.4
$k = 2$	92.3	93.5	1,426	20.2
$k = 4$	94.7	87.2	1,549	19.8
$k = 8$	95.8	78.3	1,573	19.6
$k = 16$	96.4	68.7	1,581	19.7

Table 9: Ablation study on compression algorithm components.

Compression Scheme	Ratio	Perplexity
None (FP16)	1.00×	3.32
INT8 Only	2.00×	3.34 (+0.6%)
INT8 + Delta	2.73×	3.35 (+0.9%)
INT8 + Delta + RLE	3.21×	3.36 (+1.2%)
<i>Benefit of Delta</i>	<i>+36%</i>	<i>+0.3%</i>
<i>Benefit of RLE</i>	<i>+18%</i>	<i>+0.3%</i>

FPGA power breakdown is: 28W for cache engine logic, 12W for CXL controllers, and 6W for auxiliary components (per device). The compression/decompression pipelines are particularly efficient, consuming only 8W despite processing 400GB/s throughput.

4.7 Ablation Studies

We conduct ablation studies to understand the contribution of individual components.

4.7.1 Prefetch Aggressiveness. Table 8 shows the impact of prediction horizon k on performance.

The optimal $k = 4$ provides the best balance: higher k shows diminishing returns in hit rate while significantly reducing precision (wasting bandwidth). Lower k achieves high precision but misses prefetch opportunities, resulting in lower throughput.

4.7.2 Compression Algorithm Components. Table 9 shows the contribution of each compression technique.

Delta encoding contributes significantly to compression ratio (+36%) with minimal accuracy cost (+0.3% perplexity). RLE provides additional benefit (+18%) also with minimal cost. The full pipeline

achieves $3.21\times$ compression with 1.2% perplexity degradation—an excellent trade-off.

5 Conclusion

The massive memory requirements of KV-caches fundamentally limit LLM serving throughput and cost-effectiveness. We introduced CXL-SpecKV, combining CXL memory disaggregation, FPGA acceleration, and speculative prefetching to overcome this memory wall.

Our evaluation demonstrates: $3.2\times$ throughput improvement, $8\times$ memory capacity expansion ($24\times$ with compression), 8.2% latency overhead, $2.3\times$ better cost-performance, and $1.90\times$ energy efficiency.

CXL-SpecKV’s key innovations include: (i) first CXL architecture for LLM KV-caches with coherent memory pooling, (ii) FPGA cache engine achieving 800MHz with 30% ALM utilization, (iii) lightweight LSTM prefetcher with 95% accuracy and $<10\mu\text{s}$ latency, and (iv) seamless framework integration.

Future directions include: dynamic activation sparsity in attention mechanisms, multi-tenant workload management with QoS guarantees, adaptation for 32K-128K context windows using hierarchical prediction, extension to fine-tuning/training with bi-directional patterns, leveraging CXL 3.0’s 128GB/s bandwidth and memory-side caching, heterogeneous memory tiers (HBM/DDR/NVM), security and isolation for shared CXL memory, and comparative analysis of alternative interconnects (NVLink-C2C, Infinity Fabric).

CXL-SpecKV demonstrates that memory disaggregation, when optimized for AI workloads, can match monolithic designs while providing superior flexibility. Our work highlights the importance of co-designing compute and memory subsystems, and shows reconfigurable hardware’s unique advantages for complex memory management. The success of speculative prefetching reveals exploitable patterns in LLM generation, opening opportunities for predictive optimization across AI serving stacks.

As LLMs scale, disaggregated architectures with hardware acceleration and intelligent prefetching will be critical for sustainable AI infrastructure, and our work delivers a cost-efficient FPGA-accelerated disaggregated speculative KV-cache system for next-generation datacenter LLM serving.

References

- [1] Marcos K Aguilera, Nadav Amit, William J Bolosky, Atul Chaugule, Camille Coppens, Jérémie Duchesne, Chaoran Guo, et al. 2019. Remote regions: a simple abstraction for remote memory. *arXiv preprint arXiv:1908.06189* (2019).
- [2] Meta AI. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. (2023). arXiv:2305.13245 [cs.CL] <https://arxiv.org/abs/2305.13245>
- [4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, et al. 2022. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. *arXiv preprint arXiv:2207.00032* (2022).
- [5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).
- [6] Silas Boyd-Wickizer and Jintao Zhai. 2021. Memory controller design for FPGA-based systems. In *Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 102–113.
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural*

- information processing systems 33 (2020), 1877–1901.
- [8] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774* (2024).
 - [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
 - [10] ShareGPT Community. 2023. ShareGPT: Share your wildest ChatGPT conversations. <https://sharegpt.com>.
 - [11] CXL Consortium. 2022. Compute Express Link Specification Revision 2.0.
 - [12] Intel Corporation. 2024. Intel Agilex 7 FPGA Architecture. (2024).
 - [13] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Shin. 2023. Direct access, high-performance memory disaggregation with directxl. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 287–294.
 - [14] Tae Jun Ham, Yejin Lee, et al. 2020. FPGA-based hardware acceleration of transformer attention modules. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
 - [15] Milad Hashemi, Kevin Swersky, Jamie A Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning memory access patterns. In *International Conference on Machine Learning*. 1919–1928.
 - [16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079* (2024).
 - [17] Junhyeok Jang, Hanjin Park, Jinsoo Choi, Jaehoon Lee, Suhwan Lee, et al. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2305.15838* (2023).
 - [18] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
 - [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
 - [20] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192* (2023).
 - [21] Zhe Li, Yuxuan Yang, Yiqi Xing, Zhipeng Zhang, Qingyang Wang, et al. 2024. CXL-MEM: Enabling Cost-Effective and Performant Memory Expansion for Deep Learning. *arXiv preprint arXiv:2401.13841* (2024).
 - [22] Kevin Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K Reinhardt, and Thomas F Wenisch. 2009. Disaggregated memory for expansion and sharing in blade servers. *ACM SIGARCH Computer Architecture News* 37, 3 (2009), 267–278.
 - [23] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
 - [24] Dong Liu and Yanxuan Yu. 2025. LLM EasyQuant: Scalable Quantization for Parallel and Distributed LLM Inference. (2025). *arXiv:2406.19657 [cs.LG]* <https://arxiv.org/abs/2406.19657>
 - [25] Dong Liu and Yanxuan Yu. 2025. π -Attention: Periodic Sparse Transformers for Efficient Long-Context Modeling. (2025). *arXiv:2511.10696 [cs.CL]* <https://arxiv.org/abs/2511.10696>
 - [26] Dong Liu and Yanxuan Yu. 2025. TinyServe: Query-Aware Cache Selection for Efficient LLM Serving. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 12529–12537.
 - [27] Dong Liu, Yanxuan Yu, and Ben Lengerich. 2025. CSV-Decode: Certifiable Sub-Vocabulary Decoding for Efficient Large Language Model Inference. *arXiv preprint arXiv:2511.21702* (2025).
 - [28] Dong Liu, Yanxuan Yu, Xuhong Wang, Ben Lengerich, and Ying Nian Wu. [n. d.]. MKA: Memory-Keyed Attention for Efficient Long-Context Reasoning. In *ICML 2025 Workshop on Long-Context Foundation Models*.
 - [29] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).
 - [30] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Wong, Yaohui Zhu, and Zhihao Jia. 2024. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781* (2024).
 - [31] Ramesh Nallapati, Bowen Zhou, Caglar Gulcehre, Bing Xiang, et al. 2016. Abstractive text summarization using sequence-to-sequence rnns and beyond. In *Proceedings of The 20th SIGNLL Conference on Computational Natural Language Learning*. 280–290.
 - [32] NVIDIA. 2023. FasterTransformer: Transformer Inference Library. (2023).
 - [33] OpenAI. 2023. GPT-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
 - [34] Tirthak Patel, Sunita Potluri, and Devesh Tiwari. 2020. Prefetching for CNNs: Exploiting spatial and temporal locality in modern DNN inference. In *2020 IEEE International Conference on Big Data (Big Data)*. 1533–1542.
 - [35] Hongyi Peng, Sitao Huang, Tong Zhou, et al. 2023. FPGA Acceleration of Transformer-based Large Language Models. In *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. 84–94.
 - [36] Jiantao Qiu, Jie Wang, Song Yao, Kaiyuan Guo, Boxun Li, Erjin Zhou, Jincheng Yu, Tianqi Tang, Ningyi Xu, Sen Song, et al. 2016. Going deeper with embedded fpga platform for convolutional neural network. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 26–35.
 - [37] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2383–2392.
 - [38] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).
 - [39] Zheng Ruan, Malte Schwarzkopf, Marcos K Aguilera, and Adam Belay. 2023. POND: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 574–587.
 - [40] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiyang Zhang. 2018. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 69–87.
 - [41] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. *arXiv preprint arXiv:2303.06865* (2023).
 - [42] Shriram Swamy, Jian Liu, et al. 2020. FPGA-based memory controllers for non-volatile memory systems. *IEEE Trans. Comput.* 69, 8 (2020), 1145–1158.
 - [43] Endri Taka, Ning-Chi Huang, Chi-Chih Chang, Kai-Chiang Wu, Aman Arora, and Diana Marculescu. 2025. Systolic Sparse Tensor Slices: FPGA Building Blocks for Sparse and Dense AI Acceleration. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 159–171.
 - [44] Gemma Team. 2024. Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295* (2024).
 - [45] Qwen Team. 2024. CodeQwen1.5: Powering Code Intelligence at Scale. *arXiv preprint (2024)*.
 - [46] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrutai Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
 - [47] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438* (2023).
 - [48] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).
 - [49] AMD Xilinx. 2023. Versal AI Core Series Architecture Manual. (2023).
 - [50] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
 - [51] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. 2015. Optimizing fpga-based accelerator design for deep convolutional neural networks. In *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 161–170.
 - [52] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048* (2023).